FileRerankingBench: A Benchmark for File Selection in Code-Editing Agents

Ahmad Jiha Daniel Chen
Anything, Inc.

Abstract

Code-editing agents must often choose which project files to inspect or edit before they can complete a user request. That file-selection step is easy to hide inside larger end-to-end agent evaluations, but failures at this step can dominate the user-visible behavior of a code-editing system: if the relevant file is absent from context, even a strong editor may fail. FileRerankingBench isolates this pre-edit ranking problem. Each benchmark case contains a user prompt, a candidate file list with timestamp metadata, a project corpus, and prompt-level relevance labels. The first public release is intentionally small: five query fixtures over one hiking-app project. This paper specifies the task, data format, labeling semantics, evaluation metrics, reference baselines, published aggregate results, and intended use of the public artifact. The release is limited to benchmark data, aggregate numbers, and paper materials.

1 Introduction

Large language model agents for software editing do not usually operate over a complete repository at every turn. Project files must be selected, ranked, or otherwise compressed before the editor receives context. This creates a retrieval problem that is adjacent to code search but not identical to it. The query is often a natural-language product request rather than an API name. The desired output is not a document to read, but one or more files likely to be edited. The cost of a miss is high: when the relevant file is excluded, the downstream editor may modify an adjacent file, create redundant code, or spend extra turns recovering information that could have been available immediately.

Many code-agent benchmarks evaluate final task success. That is valuable, but it entangles several abilities: understanding the user request, retrieving the right files, editing code, preserving existing behavior, and satisfying tests. FileRerankingBench focuses on one narrower question: given a user request and a candidate file set, can a ranking method put the relevant file paths near the top? This isolated formulation makes it possible to compare simple ranking rules, metadata priors, and future learned rankers without requiring a full agent run.

The current public release should be read as a benchmark artifact and format definition rather than a broad statistical study. It contains one project and five query fixtures. That scale is too small to support claims about general model quality, but it is large enough to define the task precisely, publish reference metrics, and provide regression cases for future dataset expansion. Keeping the first release small also makes the data easy to inspect manually, which is useful for checking path conventions, empty-label behavior, timestamp handling, and metric definitions.

This paper contributes:

- a formal definition of prompt-conditioned file reranking;

- a public JSON fixture format for candidate files, timestamps, and relevance labels;
- a small project corpus and five labeled query fixtures;
- Recall@k and Precision@k definitions, including edge cases for empty relevance sets and candidate lists shorter than k;
- transparent reference numbers for simple baselines; and
- a data card describing intended use, non-goals, and known limitations.

2 Task Definition

Let P be a project corpus containing source files and related project metadata. A benchmark case supplies a user prompt q, a candidate file set $C = \{c_1, \ldots, c_n\}$, and a set of expected relevant file paths R. Each candidate c_i contains at least a path, a creation timestamp, and an optional update timestamp. A ranking method receives q and C and returns an ordered list of candidate file paths. The objective is to place files in R near the top of that ordered list.

The benchmark evaluates the file-selection stage before editing. It does not score whether a generated patch is correct, whether a user interface looks good, or whether tests pass after a change. Those are important questions, but they belong to larger end-to-end evaluations. FileRerankingBench deliberately removes the editor from the loop so that retrieval errors are visible on their own.

2.1 Candidate Files

Candidate files are the files made available to a ranking method for a single query. The current fixtures use three candidate files per query, all from the same small web app. Future fixtures may include larger candidate sets, generated boilerplate, tests, configuration files, or files from several app surfaces. Candidate metadata is part of the task because real file-selection decisions often depend on both content and recency. A method may ignore metadata, use it as a tie breaker, or rank primarily from it.

Candidate paths in the JSON file are app-root paths with a leading slash, such as `/apps/web/pages/index/+Page.jsx`. The project zip stores files below `createxyz-project/`. To resolve a fixture path inside the zip, remove the leading slash and join the result below that root directory. This convention keeps labels stable even if the archive is unpacked into a different local directory.

2.2 Relevance Labels

The label set R contains file paths expected to be relevant for satisfying the prompt. In the first release, relevance is single-file for four cases and empty for one negative case. An empty relevance set means the requested target is not present in the candidate set. This is a meaningful file-selection condition: a ranking method should not be rewarded for confidently selecting an unrelated file when the prompt asks for a page that does not exist.

The benchmark allows multi-file labels even though the first release does not exercise them. Multi-file labels are important for future cases where a request requires coordinated edits, such as changing a component and its caller or adding behavior that spans route, state, and test files. Metrics in Section 4 are defined for arbitrary $|R|$.

3 Dataset Design

FileRerankingBench stores benchmark cases as a JSON array. Each element has a single `input` object. Table 1 lists the fields used in the current release.

Field	Description
`input.project`	Project corpus identifier. In the current release this is `hiking-app-small`.
`input.userPrompt`	Natural-language request that requires selecting zero or more relevant files.
`input.files[]`	Candidate files for the query. Each item includes `path`, `createdAt`, and `updatedAt`.
`input.expectedRelevantFiles[]`	Paths labeled relevant for the prompt. The array may be empty when no candidate file is relevant.

Table 1: Public fixture schema. The schema is intentionally simple so that external tools can load the benchmark without adopting a specific agent stack.

The first corpus is a hiking-themed app with three candidate page files:

- `/apps/web/pages/index/+Page.jsx`
- `/apps/web/pages/hiking-history/+Page.jsx`
- `/apps/web/pages/explore/+Page.jsx`

The project archive also contains surrounding app files, package files, and mobile-app scaffolding. The benchmark cases restrict the candidate set to the three web pages above so that the first release stays manually inspectable. A larger candidate set is a priority for subsequent releases.

3.1 Query Types

The five public fixtures are designed to cover a few distinct retrieval conditions rather than many independent applications. Table 2 summarizes the cases.

4 Metrics

The benchmark reports Recall@k and Precision@k for $k \in \{1, 3, 5, 10\}$. Let T_k be the first k files returned by a ranking method after truncating to the available candidates, and let R be the expected relevance set.

$$\text{Recall@}k = \begin{cases} 1 & \text{if } |R| = 0, \\ \frac{|T_k \cap R|}{|R|} & \text{otherwise.} \end{cases}$$

$$\text{Precision@}k = \begin{cases} 0 & \text{if } |T_k| = 0, \\ \frac{|T_k \cap R|}{|T_k|} & \text{otherwise.} \end{cases}$$

The empty-relevance recall convention deserves emphasis. If no candidate is relevant, recall is defined as 1 because there is no relevant file to retrieve. Precision remains sensitive to the ranked output: selecting unrelated files does not create true positives. This mirrors the practical distinction between not missing a relevant file and avoiding false confidence in irrelevant files.

Scores are macro-averaged across query fixtures. Macro averaging gives every prompt equal weight, which is appropriate for a small benchmark where each fixture is hand-authored and

Type	Prompt pattern	Expected file behavior
Direct page reference	The prompt names the landing page and a button on that page.	The index page should rank first.
Direct reference with recency	The same landing-page request, but timestamp metadata favors the same target.	The index page remains relevant.
Paraphrased UI reference	The prompt names button text without naming the file.	The hiking-history page should rank first.
Paraphrase with misleading recency	The same button-text request, but another page has fresher metadata.	Content should beat misleading recency.
Missing target	The prompt asks for a Scoreboard page not present in the candidate set.	No candidate file is relevant.

Table 2: Query conditions in the first public release. The dataset is small, but it includes lexical, metadata, paraphrase, and negative-label behavior.

intended to represent a distinct condition. Future larger releases may also report project-level averages or stratified metrics by query type.

When a query has fewer than k candidate files, T_k contains all available candidates. In the current release every query has three candidates, so Recall@5 and Recall@10 are identical to Recall@3; likewise Precision@5 and Precision@10 use the same three returned files. The metrics are still reported at all four cutoffs so that result tables remain compatible with future fixtures that contain larger candidate sets.

5 Signal Definitions

The benchmark is intentionally agnostic about how a ranked list is produced, but file-selection systems are often easiest to compare when described as signals. A signal is a deterministic or learned scoring rule that maps a prompt q and candidate file c_i to either a real-valued score or a rank. Signals can be evaluated independently, normalized onto a common scale, or combined into a fused ranking. This section defines signal families that are abstract enough to reproduce from public data and broad enough to cover simple baselines as well as future submissions.

5.1 Lexical Matching

A lexical signal scores candidates by token overlap between the prompt and file evidence. The evidence may include the path, filename, directory names, and file contents. Let $\tau(x)$ be a tokenizer that lowercases text, splits words, and removes punctuation. A simple overlap score is:

$$s_{\text{lex}}(q, c_i) = \left|\tau(q) \cap \tau(\text{path}(c_i) \cup \text{text}(c_i))\right| .$$

More sophisticated sparse-retrieval variants, such as BM25 [3], use term frequency, inverse document frequency, and document-length normalization. These variants are still lexical signals under this benchmark: they rely on observed tokens rather than learned semantic representations. Lexical matching is transparent and cheap, but it can miss paraphrases. In the current corpus, a button-text prompt can point to a file whose path does not share the strongest surface words with the request.

5.2 Semantic Similarity

A semantic similarity signal embeds the prompt and each candidate into a vector space and ranks by a similarity function such as cosine similarity:

$$s_{\text{sem}}(q, c_i) = \frac{e(q) \cdot e(c_i)}{\|e(q)\|\|e(c_i)\|}.$$

Here $e(\cdot)$ is any documented embedding function over natural language and code text. A result that uses this signal should specify the embedding model, the file text included in the embedding, truncation behavior, and whether paths are embedded alongside contents. The benchmark does not prescribe a specific embedding model. The definition is useful because it separates the signal family from any particular model choice.

5.3 Learned Pairwise Reranking

A learned reranking signal scores a prompt-file pair directly:

$$s_{\text{rank}}(q, c_i) = g(q, \text{path}(c_i), \text{text}(c_i)),$$

where g may be a cross-encoder, classifier, preference model, or other learned function. Unlike a bi-encoder semantic-similarity signal, a pairwise reranker can jointly attend to prompt and file evidence. To be reproducible, submissions using this family should state the model, input construction, maximum file length, and how unavailable or overlong file text is handled. The public benchmark need not know those details to define the task, but the details matter for comparing independent results.

5.4 Recency

A recency signal uses timestamp metadata rather than file text. Define

$$t(c_i) = \begin{cases} \text{updatedAt}(c_i) & \text{if present,} \\ \text{createdAt}(c_i) & \text{otherwise.} \end{cases}$$

Candidates are sorted by $t(c_i)$ descending. The sorted position can be used directly as a rank, or converted to a score with a reciprocal-rank transform:

$$s_{\text{rec}}(c_i) = \frac{1}{r_i + \alpha},$$

where r_i is the zero-based recency rank and $\alpha > 0$ is a smoothing constant. Recency is reproducible from fixture metadata alone. It is also a good example of why metadata should be part of a file-selection benchmark: timestamp signals can help on follow-up edits but can also mislead when a fresher file is not relevant to the prompt.

5.5 Structural Context

Some file-selection settings include structural context beyond the prompt and candidate list: the file currently being viewed, an import graph, route relationships, or component ownership information. A structural signal assigns score from those relationships. For example, if o is a known open file and $d(o, c_i)$ is graph distance from o to candidate c_i, then a decayed proximity score can be written as:

$$s_{\text{struct}}(c_i) = \begin{cases} \lambda^{d(o,c_i)} & \text{if } c_i \text{ is reachable from } o, \\ 0 & \text{otherwise,} \end{cases}$$

with $0 < \lambda < 1$. The current fixture schema does not include an open-file field or graph annotations, so this signal is not part of the published reference baselines. It is included here because the definition is general and future benchmark releases may include explicit structural context.

5.6 Open-File Signal Imputation

Open-file signal imputation is a specific structural signal for edit-follow-up turns. The motivating observation is that a short prompt such as "make this smaller" may not contain enough lexical information to identify a file, but the file currently in view is strong context. The signal starts with a seed score on the open file and imputes relevance to nearby files through a public structural relation such as an import graph.

Let o be the open file, $G = (C, E)$ be a directed graph over candidate files, and $\mathcal{N}(u)$ be the outgoing neighbors of file u. One reproducible definition initializes:

$$s_{\text{imp}}(o) = 1, \qquad s_{\text{imp}}(c_i) = 0 \text{ for } c_i \neq o.$$

For each reachable edge (u, v), score can be propagated with decay:

$$s_{\text{imp}}(v) = \max\left(s_{\text{imp}}(v), \frac{\lambda \cdot s_{\text{imp}}(u)}{\max(1, |\mathcal{N}(u)|)}\right),$$

where $0 < \lambda < 1$. The division by out-degree prevents high-fanout files from giving every dependency the same confidence as the open file, while the `max` rule keeps the strongest path when a file is reachable in multiple ways. A submission could instead use summation or a different graph relation, but it should publish the seed file source, graph construction, decay rule, cycle handling, and whether scores are propagated along imports, reverse imports, routes, component references, or another relation.

The current public fixtures do not include open-file metadata, so this signal is defined for reproducibility and future dataset growth rather than scored in the first reference table.

5.7 Boilerplate and Distractor Priors

Generated apps and framework templates often contain files that are valid code but unlikely edit targets for user-facing requests. A boilerplate or distractor prior assigns lower score to files known to be scaffolding and higher score to files known to be authored or task-relevant candidates:

$$s_{\text{prior}}(c_i) = \begin{cases} 1 & \text{if } c_i \in A, \\ 0 & \text{if } c_i \in B, \end{cases}$$

where A is a set of candidate files considered authored or otherwise eligible, and B is a set considered boilerplate or distractor-heavy. More graded priors are also possible. To keep this signal reproducible, any such sets should be derived from public fixture metadata or published annotations rather than private project knowledge.

5.8 Signal Normalization and Fusion

Signals may produce scores on different scales. A lexical score may be an integer overlap count, a sparse retriever may produce unbounded positive numbers, a semantic signal may produce cosine values, and recency may produce a rank. To combine signals, a benchmark runner can normalize each score-valued signal with min-max normalization:

$$\tilde{s}_j(c_i) = \frac{s_j(c_i) - \min_{c \in C} s_j(c)}{\max_{c \in C} s_j(c) - \min_{c \in C} s_j(c)}.$$

If the denominator is zero, the signal does not distinguish candidates for that query and can be assigned a constant score. Rank-valued signals can be converted with reciprocal-rank scoring as above. A fused score is then:

$$S(c_i) = \sum_{j=1}^{m} w_j \tilde{s}_j(c_i),$$

where $w_j \geq 0$ and $\sum_j w_j = 1$. A result using fusion should publish the signal definitions, weights, normalization rules, and tie-breaking policy. Those choices are part of the evaluated method, not hidden benchmark state.

6 Reference Baselines

The public result table includes four reference rows. These rows are not meant to represent a complete leaderboard. They are sanity checks that make the metric definitions concrete and give future submissions a stable point of comparison. They also instantiate a subset of the signal families above: original order uses no scoring signal, recency uses timestamp metadata, lexical matching uses prompt-file token evidence, and the oracle row exposes the label-induced upper bound for the current candidate sets.

Original candidate order. This baseline returns files in the order provided by the fixture. It is useful because JSON order is visible, deterministic, and requires no scoring model. If a benchmark case is accidentally ordered with the answer first, this row makes that bias visible.

Recency baseline. This baseline ranks candidates by the most recent timestamp available for each file, using `updatedAt` when present and `createdAt` otherwise. It tests whether timestamp metadata alone is enough to recover the relevant file. In the current fixture set, recency helps in one direct-reference case but hurts in the paraphrased case where the fresher file is not the target.

Lexical baseline. This baseline ranks by transparent lexical overlap between the user prompt and candidate file evidence. It is intended to capture the simplest keyword-matching strategy: distinctive words in the prompt should help when they appear in the target path or file text, while paraphrases and absent pages remain difficult.

Rank-all oracle upper bound. This row places all labeled relevant files before non-relevant files. It is not a deployable method; it is an upper bound induced by the labels and candidate set. The row is useful for checking metric behavior, especially the effect of the empty-label query on macro-averaged precision.

7 Published Results

Table 3 reports aggregate reference numbers over the current public fixture set. The table should be interpreted as a benchmark-format anchor and regression reference. It should not be interpreted as evidence that one general file-selection approach is superior across broad code-editing tasks.

System	R@1	P@1	R@3	P@3	R@5	P@5	R@10	P@10
Original candidate order	0.6000	0.4000	1.0000	0.2667	1.0000	0.2667	1.0000	0.2667
Recency baseline	0.4000	0.2000	1.0000	0.2667	1.0000	0.2667	1.0000	0.2667
Lexical baseline	0.6000	0.4000	1.0000	0.2667	1.0000	0.2667	1.0000	0.2667
Rank-all oracle upper bound	1.0000	0.8000	1.0000	0.2667	1.0000	0.2667	1.0000	0.2667

Table 3: Macro-averaged results over five public benchmark cases. Because each case has three candidate files, the @5 and @10 columns match the @3 columns in this release.

The numbers expose two properties of the first dataset. First, the top-1 position matters: all non-oracle baselines reach Recall@3 because all available files are included by that point, but their Recall@1 and Precision@1 differ. Second, the negative query lowers precision for rank-all-style outputs. The rank-all oracle has Recall@1 of 1.0 because it retrieves the relevant file when one exists and has no relevant file to miss in the empty-label case; its Precision@1 is 0.8 because the empty-label case contributes no true positive.

8 Data Card

Dataset name. FileRerankingBench, first public release.

Dataset contents. The release contains `data/file-reranking-evals.json`, one project corpus archive at `data/projects/hiking-app-small.zip`, aggregate result tables, and this paper. The fixture file contains five query cases. Each case references the same project and provides three candidate files.

Task. Given a prompt and candidate file metadata, return a ranked list of candidate file paths. The task is evaluated against prompt-level relevance labels.

Intended use. The dataset is intended for small-scale regression testing, metric validation, format validation, and demonstration of file-selection evaluation. It can also serve as seed data for larger file-reranking datasets.

Out-of-scope use. The dataset is not large enough to support broad claims about model quality, agent quality, or general code-search performance. It should not be used as the sole basis for choosing a ranking method.

Privacy and licensing. The project corpus is included for public benchmark use. The release does not include private repositories, user data, or source code for ranking systems.

Known biases. The first release is web-page-heavy, English-only, and centered on a single small app. Candidate sets are tiny compared with real projects. Labels are single-file except for the negative case. These choices make the first release easy to inspect, but they limit external validity.

9 Relationship to Broader Evaluations

End-to-end software-agent evaluations such as repository repair benchmarks measure whether a model can produce a final correct change. FileRerankingBench occupies a smaller slice of that problem. It asks whether the right files are available near the top of the candidate list before editing begins. This positioning is complementary: a file-selection benchmark cannot replace end-to-end tests, but end-to-end tests can make retrieval failures difficult to diagnose.

The benchmark is also related to information retrieval metrics such as BM25 and top-k recall [3]. The difference is the unit of retrieval. In web or passage retrieval, the returned item is usually the final object the user consumes. In file selection for code editing, the returned item is intermediate context for another process. That makes high recall at small k particularly important: the file need only be available to the editor, but if it is absent, later stages may never recover.

Long-context work has shown that simply providing more text is not always a solution; models may underuse information placed in long inputs [2]. File reranking is therefore not only a cost optimization. It is also a way to control which information receives attention. A benchmark that isolates this step can help compare whether a method finds the relevant files before the downstream editor consumes context.

10 Limitations and Future Work

The most important limitation is size. Five query fixtures over one project are not enough for a broad empirical claim. The current release is best understood as a public seed and a precise specification. The next release should add more projects, larger candidate sets, multi-file labels, and more negative cases.

The second limitation is task diversity. The current corpus is focused on page selection in a small web app. Future versions should include mobile files, backend routes, shared components, tests, configuration files, data schemas, and cross-file edits. These additions would make the benchmark more representative of the range of files a code-editing agent may need.

The third limitation is label depth. The current labels identify relevant file paths, not spans, edit locations, or reasons for relevance. Path-level labels are the right starting point for file reranking, but richer annotations could support more detailed analysis. For example, future releases could distinguish "must edit," "useful context," and "distractor" files.

Finally, the current benchmark reports only aggregate metrics. As the dataset grows, result reports should include per-category breakdowns: lexical vs. paraphrased prompts, recency-helpful vs. recency-misleading cases, single-file vs. multi-file labels, and present-target vs. absent-target prompts. Those breakdowns will make it easier to see which retrieval conditions are actually improving.

11 Release Contents

The public repository at `https://github.com/Create-Inc/file-reranking-bench` contains:

- `data/file-reranking-evals.json`: benchmark fixture JSON;
- `data/projects/hiking-app-small.zip`: project corpus archive;
- `results/aggregate-results.csv`: aggregate reference numbers;
- `RESULTS.md`: human-readable result summary; and
- `paper.tex` and `paper.pdf`: this paper.

The release is deliberately limited to benchmark artifacts and published aggregate numbers. It does not include source code for ranking systems or runtime details.

12 Conclusion

FileRerankingBench provides a compact public artifact for evaluating the file-selection step in code-editing agents. The first release is small, but it defines a concrete task, a simple data schema, metric behavior for important edge cases, and reference numbers for transparent baselines. The benchmark is intended to grow into a larger corpus while preserving the same public contract: prompt-level labels, candidate file metadata, project corpora, and aggregate evaluation results.

References

[1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[2] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[3] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.